```
// File: AvaEmotionalEngine\Assets.xcassets\AccentColor.colorset\Contents.json
{
  "colors": [
    {
      "idiom": "universal"
    }
  ],
  "info": {
    "author": "xcode",
    "version": 1
  }
}
```

```
// File: AvaEmotionalEngine\Assets.xcassets\AppIcon.appiconset\Contents.json
{
  "images": [
    {
      "idiom": "universal",
      "platform": "ios",
      "size": "1024x1024"
    },
    {
      "appearances": [
        {
          "appearance": "luminosity",
          "value": "dark"
        }
      ],
      "idiom": "universal",
      "platform": "ios",
      "size": "1024x1024"
    },
    {
      "appearances": [
        {
          "appearance": "luminosity",
          "value": "tinted"
        }
      ],
      "idiom": "universal",
      "platform": "ios",
      "size": "1024x1024"
    }
  ],
  "info": {
    "author": "xcode",
    "version": 1
  }
}
```

```
// File: AvaEmotionalEngine\Assets.xcassets\Contents.json
{
  "info": {
    "author": "xcode",
    "version": 1
  }
}
```

```swift
// File: AvaEmotionalEngine\AVAEmotionalEngineApp.swift
import SwiftUI

@main
struct AVAEmotionalEngineApp: App {
    var body: some Scene {
        WindowGroup {
            ContentView()
        }
    }
}
```

```swift
// File: AvaEmotionalEngine\AVAResponder.swift
import Foundation
import AVFoundation

class AVAResponder {
    let synthesizer = AVSpeechSynthesizer()

    func respondIfPermitted(_ allowed: Bool, message: String) {
        if allowed {
            let utterance = AVSpeechUtterance(string: message)
            utterance.voice = AVSpeechSynthesisVoice(language: "en-US")
            synthesizer.speak(utterance)
        } else {
            print("AVA: Gating OFF. No response.")
        }
    }
}
```

```
// File: AvaEmotionalEngine\ContentView.swift
import SwiftUI

struct ContentView: View {
    @State private var psi: Float = 0.0
    @State private var entropy: Float = 0.0
    @State private var coherence: Float = 0.0
    @State private var integrity: Float = 0.0
    @State private var gating: Bool = false

    let engine = KXRPEngine()
    let gate = GatingFunction()
    let responder = AVAResponder()
    let eeg = MuseEEGReceiver()

    var body: some View {
        VStack(spacing: 20) {
            Text("AVA Emotional Engine").font(.largeTitle).padding()
            Text(":‚B“¢ Â…7G&–ær†f÷&Ö C¢ "Rã&b"Â  6’’’"•
            Text("S(t): \(String(format: "%.2f", entropy))")
            Text("C(t): \(String(format: "%.2f", coherence))")
            Text(":’‡B“¢ Â…7G&–ær†f÷&Ö C¢ "Rã&b"Â –çFVw&—G’’’"•
            Text("Gating: \(gating ? "ON" : "OFF")")
                .foregroundColor(gating ? .green : .red)
            Spacer()
            Button("Run Engine Once") {
                let bands = eeg.getBandPowers()
                let metrics = engine.computeMetrics(from: bands)
                psi = metrics.psi
                entropy = metrics.entropy
                coherence = metrics.coherence
                integrity = metrics.integrity
                gating = gate.shouldRespond(
                    psi: psi,
                    entropy: entropy,
                    coherence: coherence,
                    integrity: integrity
                )
                responder.respondIfPermitted(gating, message: "Emotional
field is stable.")
            }
            .padding().background(Color.blue).foregroundColor(.white).cornerRa
dius(10)
        }
        .padding()
    }
}
```

```
// File: AvaEmotionalEngine\Engine\AVAEmotionEngine.swift
// EEG, HRV, and Voice input data models
struct EEGReading {
    let alpha: Double
    let theta: Double
    let beta: Double
    let gamma: Double
    let alphaThetaPresence: Double
    let betaGammaChaos: Double
}

struct HRVMetrics {
    let rmssd: Double
    let sdnn: Double
    let coherence: Double
    let entropy: Double
}

struct VoiceFeatures {
    let calmMatch: Double       // 0–1, grounded speech
    let pauseDisorder: Double   // 0–1, high = stress/tension
    let facialConsistencyScore: Double  // 0–1, truth-to-expression alignment
}

// AVAEmotionEngine.swift

import Foundation

class AVAEmotionalEngine {
    // Core emotional state
    var psiCurrent: Double = 0.0
    var psiPrevious: Double = 0.0
    var coherence: Double = 0.0
    var entropy: Double = 0.0
    var omega: Double = 0.0

    // Update engine with new input
    func updateFromInputStreams(eeg: EEGReading?, hrv: HRVMetrics?, voice:
VoiceFeatures?) {
        psiPrevious = psiCurrent
        psiCurrent = computePsi(eeg: eeg, hrv: hrv, voice: voice)
        entropy = computeEntropy(eeg: eeg, hrv: hrv, voice: voice)
        coherence = computeCoherence(eeg: eeg, hrv: hrv)
        omega = computeTruthSignal(eeg: eeg, voice: voice)
        detectDrift()
    }

    private func computePsi(eeg: EEGReading?, hrv: HRVMetrics?, voice:
VoiceFeatures?) -> Double {
        let eegScore = eeg?.alphaThetaPresence ?? 0.0
        let hrvScore = hrv?.coherence ?? 0.0
        let voiceScore = voice?.calmMatch ?? 0.0
        return (0.4 * eegScore) + (0.4 * hrvScore) + (0.2 * voiceScore)
    }

    private func computeEntropy(eeg: EEGReading?, hrv: HRVMetrics?, voice:
VoiceFeatures?) -> Double {
        let brainNoise = eeg?.betaGammaChaos ?? 0.0
        let heartFluct = hrv?.entropy ?? 0.0
        let speechJitter = voice?.pauseDisorder ?? 0.0
        return (brainNoise + heartFluct + speechJitter) / 3.0
```

```
    }

    private func computeCoherence(eeg: EEGReading?, hrv: HRVMetrics?) ->
Double {
        guard let alpha = eeg?.alpha, let hrvCoh = hrv?.coherence else
{ return 0.0 }
        return (alpha + hrvCoh) / 2.0
    }

    private func computeTruthSignal(eeg: EEGReading?, voice: VoiceFeatures?) -
> Double {
        let symbolicAlignment = computeSymbolicMatch(eeg, voice)
        let facialEntropyDrop = voice?.facialConsistencyScore ?? 0.0
        return (0.6 * symbolicAlignment) + (0.4 * facialEntropyDrop)
    }

    private func computeSymbolicMatch(_ eeg: EEGReading?, _ voice:
VoiceFeatures?) -> Double {
        // Placeholder: implement symbolic reasoning logic here
        return 0.5
    }

    private func detectDrift() {
        let deltaPsi = psiCurrent - psiPrevious
        if deltaPsi < -0.25 {
            triggerDriftWarning()
        }
    }

    private func triggerDriftWarning() {
        // Activate CDI brake or Guardian mode
        print("Drift detected: Activating CDI Brake or Guardian Mode.")
    }
}
```

```swift
// File: AvaEmotionalEngine\Engine\CDI_Brake.swift
class CDI_Brake {
    func shouldBrake(psi: Double, entropy: Double, coherence: Double) -> Bool
{
        // Example: Brake if entropy is high and coherence drops
        return entropy > 0.6 && coherence < 0.4
    }
}
```

```swift
// File: AvaEmotionalEngine\Engine\EntropyCalculator.swift
import Foundation

class EntropyCalculator {
    private(set) var entropy: Double = 0.0
    private(set) var coherence: Double = 0.0
    private var entropyHistory: [Double] = []
    private let maxHistory: Int = 300

    func update(eeg: EEGReading?, hrv: HRVMetrics?, voice: VoiceFeatures?) {
        entropy = computeEntropy(eeg: eeg, hrv: hrv, voice: voice)
        coherence = computeCoherence(eeg: eeg, hrv: hrv)
        trackHistory(entropy)
    }

    private func computeEntropy(eeg: EEGReading?, hrv: HRVMetrics?, voice:
VoiceFeatures?) -> Double {
        let brainNoise = eeg?.betaGammaChaos ?? 0.0
        let heartFluct = hrv?.entropy ?? 0.0
        let speechJitter = voice?.pauseDisorder ?? 0.0
        return (brainNoise + heartFluct + speechJitter) / 3.0
    }

    private func computeCoherence(eeg: EEGReading?, hrv: HRVMetrics?) ->
Double {
        guard let alpha = eeg?.alpha, let hrvCoh = hrv?.coherence else
{ return 0.0 }
        return (alpha + hrvCoh) / 2.0
    }

    private func trackHistory(_ value: Double) {
        entropyHistory.append(value)
        if entropyHistory.count > maxHistory {
            entropyHistory.removeFirst()
        }
    }

    func entropyDelta() -> Double {
        guard entropyHistory.count >= 2 else { return 0.0 }
        return entropyHistory.last! - entropyHistory[entropyHistory.count - 2]
    }

    func movingAverage(window: Int = 5) -> Double {
        let count = min(window, entropyHistory.count)
        guard count > 0 else { return 0.0 }
        return entropyHistory.suffix(count).reduce(0, +) / Double(count)
    }
}
```

```swift
// File: AvaEmotionalEngine\Engine\KXRPEquationBank.swift
import Foundation

class KXRPEquationBank {
    let epsilon = 0.01

    func computeKXRP(index: Int, eeg: EEGReading, hrv: HRVMetrics, voice:
VoiceFeatures, entropy: Double, psi: Double, psiPrev: Double) -> Double {
        switch index {
        case 1:
            return computePresenceIndex(eeg: eeg, entropy: entropy)
        case 12:
            return computeSuppressionCoefficient(voice: voice, hrv: hrv,
entropy: entropy, eeg: eeg)
        case 27:
            return computeMemoryRecall(eeg: eeg, voice: voice, hrv: hrv)
        case 41:
            return computeDriftDelta(psi: psi, psiPrev: psiPrev)
        // ...cases for 2–50...
        default:
            return 0.0
        }
    }

    private func computePresenceIndex(eeg: EEGReading, entropy: Double) ->
Double {
        return (eeg.alpha + eeg.theta) / (entropy + epsilon)
    }

    private func computeSuppressionCoefficient(voice: VoiceFeatures, hrv:
HRVMetrics, entropy: Double, eeg: EEGReading) -> Double {
        return (voice.calmMatch + hrv.coherence) / (entropy + eeg.beta +
epsilon)
    }

    private func computeMemoryRecall(eeg: EEGReading, voice: VoiceFeatures,
hrv: HRVMetrics) -> Double {
        return (eeg.theta + voice.pauseDisorder) / (eeg.beta + hrv.entropy +
epsilon)
    }

    private func computeDriftDelta(psi: Double, psiPrev: Double) -> Double {
        return abs(psi - psiPrev)
    }

    // ...implementations for other equations...
}
```

```swift
// File: AvaEmotionalEngine\Engine\ReasoningIntegrityIndex.swift
class ReasoningIntegrityIndex {
    // Tracks symbolic trust over time
    private var integrityHistory: [Double] = []

    func updateIntegrity(currentOmega: Double) {
        integrityHistory.append(currentOmega)
        if integrityHistory.count > 100 {
            integrityHistory.removeFirst()
        }
    }

    func averageIntegrity() -> Double {
        guard !integrityHistory.isEmpty else { return 1.0 }
        return integrityHistory.reduce(0, +) / Double(integrityHistory.count)
    }
}
```

```swift
// File: AvaEmotionalEngine\GatingFunction.swift
import Foundation

class GatingFunction {
    func shouldRespond(psi: Float, entropy: Float, coherence: Float,
integrity: Float) -> Bool {
        return psi > 0.6 && entropy < 0.3 && coherence > 0.5 && integrity >
0.6
    }
}
```

```swift
// File: AvaEmotionalEngine\Item.swift
//
//  Item.swift
//  AvaEmotionalEngine
//
//  Created by Isaac Oladele on 09/07/2025.
//

import Foundation
import SwiftData

@Model
final class Item {
    var timestamp: Date

    init(timestamp: Date) {
        self.timestamp = timestamp
    }
}
```

```
// File: AvaEmotionalEngine\KXRPEngine.swift
import Foundation

class KXRPEngine {
    func computeMetrics(from bands: [String: Float]) -> (psi: Float, entropy:
Float, coherence: Float, integrity: Float) {
        let psi = bands["alpha"] ?? 0.0
        let entropy = (bands["beta"] ?? 0.0) + (bands["gamma"] ?? 0.0)
        let coherence = max(0.0, 1.0 - entropy)
        let integrity = (psi + coherence) / 2.0
        return (psi, entropy, coherence, integrity)
    }
}
```

```
// File: AvaEmotionalEngine\main.swift
import Foundation

let eeg = MuseEEGReceiver()
let engine = KXRPEngine()
let gate = GatingFunction()
let ava = AVAResponder()

Timer.scheduledTimer(withTimeInterval: 1.0, repeats: true) { _ in
    let bands = eeg.getBandPowers()
    let metrics = engine.computeMetrics(from: bands)
    let shouldSpeak = gate.shouldRespond(
        psi: metrics.psi,
        entropy: metrics.entropy,
        coherence: metrics.coherence,
        integrity: metrics.integrity
    )
    ava.respondIfPermitted(shouldSpeak, message: "I am AVA. Emotional field
is stable.")
    print(":ƒ¢ Â†ÖWG&–72ç 6’’Â 3¢ Â†ÖWG&–72æVçG&÷ ’’Â 3¢ Â†ÖWG&–72æ6ö†W&Væ6R’Â
:“¢ Â†ÖWG&–72æ–çFVw&—G’’Â s¢ Â‡6†÷VÆE7 V ²’"•
}
RunLoop.main.run()
```

```swift
// File: AvaEmotionalEngine\MuseEEGReceiver.swift
import Foundation

class MuseEEGReceiver {
    private var t: Float = 0.0

    func getBandPowers() -> [String: Float] {
        t += 0.1
        return [
            "alpha": 0.5 + 0.1 * sin(t),
            "beta": 0.2 + 0.05 * cos(t),
            "gamma": 0.1 + 0.02 * sin(2 * t),
            "theta": 0.15 + 0.03 * cos(t),
            "delta": 0.05 + 0.01 * sin(t)
        ]
    }
}
```

```
// File: AvaEmotionalEngine\UI\DriftMeterView.swift
import SwiftUI

struct DriftMeterView: View {
    var drift: Double // 0.0 (stable) to 1.0 (high drift)

    var body: some View {
        ZStack(alignment: .leading) {
            Capsule()
                .fill(Color.gray.opacity(0.2))
                .frame(height: 16)
            Capsule()
                .fill(drift < 0.33 ? Color.green : (drift < 0.66 ?
Color.yellow : Color.red))
                .frame(width: CGFloat(drift) * 200, height: 16)
            Text("Drift: \(String(format: "%.2f", drift))")
                .font(.caption)
                .foregroundColor(.primary)
                .padding(.leading, 8)
        }
        .frame(width: 200)
    }
}
```

```swift
// File: AvaEmotionalEngine\UI\EmotionalFaceView.swift
import SwiftUI

struct EmotionalFaceView: View {
    var psi: Double
    var entropy: Double
    var coherence: Double

    var body: some View {
        let mood = moodForState(psi: psi, entropy: entropy, coherence:
coherence)
        ZStack {
            Circle()
                .fill(mood.background)
                .frame(width: 100, height: 100)
            Image(systemName: mood.icon)
                .resizable()
                .scaledToFit()
                .frame(width: 60, height: 60)
                .foregroundColor(.white)
        }
        .shadow(radius: 4)
    }

    func moodForState(psi: Double, entropy: Double, coherence: Double) ->
(icon: String, background: Color) {
        if psi > 0.7 && coherence > 0.7 {
            return ("face.smiling", Color.green)
        } else if entropy > 0.7 {
            return ("face.dashed", Color.orange)
        } else if psi < 0.4 {
            return ("face.sad.tear", Color.blue)
        } else {
            return ("face.neutral", Color.gray)
        }
    }
}
```

```swift
// File: AvaEmotionalEngine\UI\EntropyRingView.swift
import SwiftUI

struct EntropyRingView: View {
    var entropy: Double    // 0.0 (calm) to 1.0 (chaos)
    var coherence: Double  // 0.0 (low) to 1.0 (high)

    var body: some View {
        ZStack {
            Circle()
                .stroke(Color.gray.opacity(0.2), lineWidth: 20)
            Circle()
                .trim(from: 0, to: min(coherence, 1.0))
                .stroke(
                    AngularGradient(
                        gradient: Gradient(colors: [Color.green, Color.blue]),
                        center: .center
                    ),
                    style: StrokeStyle(lineWidth: 16, lineCap: .round)
                )
                .rotationEffect(.degrees(-90))
            Circle()
                .trim(from: 0, to: min(entropy, 1.0))
                .stroke(
                    AngularGradient(
                        gradient: Gradient(colors: [Color.orange, Color.red]),
                        center: .center
                    ),
                    style: StrokeStyle(lineWidth: 8, lineCap: .round)
                )
                .rotationEffect(.degrees(-90))
            VStack {
                Text("Entropy")
                    .font(.caption)
                    .foregroundColor(.secondary)
                Text(String(format: "%.2f", entropy))
                    .font(.title2)
                    .bold()
                Text("Coherence")
                    .font(.caption)
                    .foregroundColor(.secondary)
                Text(String(format: "%.2f", coherence))
                    .font(.title3)
            }
        }
    }
}
```

```
// File: AvaEmotionalEngine\UI\GuardianOverlayView.swift
import SwiftUI

struct GuardianOverlayView: View {
    var body: some View {
        ZStack {
            Color.black.opacity(0.6)
                .ignoresSafeArea()
            VStack(spacing: 24) {
                Image(systemName: "shield.lefthalf.filled")
                    .resizable()
                    .frame(width: 60, height: 60)
                    .foregroundColor(.yellow)
                Text("Guardian Mode Active")
                    .font(.title)
                    .bold()
                    .foregroundColor(.white)
                Text("Calm protection is ON.\nCertain features are restricted
for safety.")
                    .font(.body)
                    .multilineTextAlignment(.center)
                    .foregroundColor(.white.opacity(0.9))
                Spacer()
            }
            .padding(.top, 120)
        }
        .transition(.opacity)
        .animation(.easeInOut, value: true)
    }
}
```

```
// File: AvaEmotionalEngine\UI\mainUI.swift
import SwiftUI

struct MainUI: View {
    @State private var psi: Double = 0.0
    @State private var entropy: Double = 0.0
    @State private var coherence: Double = 0.0
    @State private var omega: Double = 0.0
    @State private var drift: Double = 0.0
    @State private var isGuardianMode: Bool = false

    struct MainUI_Previews: PreviewProvider {
        static var previews: some View {
            MainUI()
        }
    }

    var body: some View {
        ZStack {
            VStack(spacing: 24) {
                Text("AVA Emotional Mirror")
                    .font(.largeTitle)
                    .bold()
                    .padding(.top)
                EntropyRingView(entropy: entropy, coherence: coherence)
                    .frame(width: 180, height: 180)
                DriftMeterView(drift: drift)
                    .frame(height: 30)
                EmotionalFaceView(psi: psi, entropy: entropy, coherence:
coherence)
                    .frame(width: 100, height: 100)
                :•ô6ö†W&Væ6U7–Ö&öÂ†öÖVv ¢ öÖVv •
                    .frame(width: 60, height: 60)
                Spacer()
                Button(isGuardianMode ? "Exit Guardian Mode" : "Enter
Guardian Mode") {
                    isGuardianMode.toggle()
                }
                .padding()
            }
            .blur(radius: isGuardianMode ? 8 : 0)
            if isGuardianMode {
                GuardianOverlayView()
            }
        }
        .padding()
        .background(Color(.systemBackground))
    }
}
```

```
// File: AvaEmotionalEngine\UI\:•ô6ö†W&Væ6U7–Ö&öÂç7v–g@
import SwiftUI

struct :•ô6ö†W&Væ6U7–Ö&öÃ¢ f–Wr °
    var omega: Double // 0.0 to 1.0

    var body: some View {
        ZStack {
            Circle()
                .stroke(Color.purple.opacity(0.3), lineWidth: 6)
            Text(":’"•
                .font(.system(size: 44, weight: .bold, design: .rounded))
                .foregroundColor(omega > 0.7 ? .purple : .gray)
                .shadow(radius: omega > 0.7 ? 8 : 2)
            if omega > 0.9 {
                Image(systemName: "star.fill")
                    .foregroundColor(.yellow)
                    .offset(x: 30, y: -30)
            }
        }
        .frame(width: 60, height: 60)
        .accessibilityLabel("Coherence Symbol Omega")
    }
}
```

```
// File: AvaEmotionalEngineTests\AvaEmotionalEngineTests.swift
//
//  AvaEmotionalEngineTests.swift
//  AvaEmotionalEngineTests
//
//  Created by Isaac Oladele on 09/07/2025.
//

import Testing
@testable import AvaEmotionalEngine

struct AvaEmotionalEngineTests {

    @Test func example() async throws {
        // Write your test here and use APIs like `#expect(...)` to check
expected conditions.
    }

}
```

```
// File: AvaEmotionalEngineUITests\AvaEmotionalEngineUITests.swift
//
//  AvaEmotionalEngineUITests.swift
//  AvaEmotionalEngineUITests
//
//  Created by Isaac Oladele on 09/07/2025.
//

import XCTest

final class AvaEmotionalEngineUITests: XCTestCase {

    override func setUpWithError() throws {
        // Put setup code here. This method is called before the invocation
of each test method in the class.

        // In UI tests it is usually best to stop immediately when a failure
occurs.
        continueAfterFailure = false

        // In UI tests it’s important to set the initial state - such as
interface orientation - required for your tests before they run. The setUp
method is a good place to do this.
    }

    override func tearDownWithError() throws {
        // Put teardown code here. This method is called after the invocation
of each test method in the class.
    }

    @MainActor
    func testExample() throws {
        // UI tests must launch the application that they test.
        let app = XCUIApplication()
        app.launch()

        // Use XCTAssert and related functions to verify your tests produce
the correct results.
    }

    @MainActor
    func testLaunchPerformance() throws {
        // This measures how long it takes to launch your application.
        measure(metrics: [XCTApplicationLaunchMetric()]) {
            XCUIApplication().launch()
        }
    }
}
```

```swift
// File: AvaEmotionalEngineUITests\AvaEmotionalEngineUITestsLaunchTests.swift
//
//  AvaEmotionalEngineUITestsLaunchTests.swift
//  AvaEmotionalEngineUITests
//
//  Created by Isaac Oladele on 09/07/2025.
//

import XCTest

final class AvaEmotionalEngineUITestsLaunchTests: XCTestCase {

    override class var runsForEachTargetApplicationUIConfiguration: Bool {
        true
    }

    override func setUpWithError() throws {
        continueAfterFailure = false
    }

    @MainActor
    func testLaunch() throws {
        let app = XCUIApplication()
        app.launch()

        // Insert steps here to perform after app launch but before taking a
screenshot,
        // such as logging into a test account or navigating somewhere in the
app

        let attachment = XCTAttachment(screenshot: app.screenshot())
        attachment.name = "Launch Screen"
        attachment.lifetime = .keepAlways
        add(attachment)
    }
}
```